Exercise 2: E-commerce Platform Search Function

CODE

```
import java.util.*;


public class EcommercePlatform {
    public static void main(String[] args) {
        Product p1 = new Product();
        Product p2 = new Product();
        Product p3 = new Product();
        Product p4 = new Product();

        p1.setProductId(15);
        p2.setProductId(72);
        p3.setProductId(34);
        p4.setProductId(58);

        p1.setProductName("Headphones");
        p2.setProductName("Webcam");
        p3.setProductName("SSD");
        p4.setProductName("RAM");

        p1.setCategory("audio");
        p2.setCategory("connectivity");
        p3.setCategory("storage");
        p4.setCategory("memory");

        List<Product> products = new ArrayList<>();

        products.add(p1);
```

```
products.add(p2);
products.add(p3);
products.add(p4);

MergeSort.sort(products);

for (Product p : products) {
   System.out.println(p.getProductId() + " ");
}
System.out.println("-------------------Array Sorted------------------");

Search ls = new LinearSearch();
Product lp = ls.searchProduct(products, 34);

if (lp == null) {
   System.out.println("Product Not Found");
} else {
   System.out.println("Product Name : " + lp.getProductName());
   System.out.println("Product Category : " + lp.getCategory());
}

Search bs = new BinarySearch();
Product bp = bs.searchProduct(products, 72);

if (bp == null) {
   System.out.println("Product Not Found");
} else {
   System.out.println("Product Name : " + bp.getProductName());
   System.out.println("Product Category : " + bp.getCategory());
```

```
        }
    }
}

class Product {
    private int productId;
    private String category;
    private String productName;

    public void setProductId(int product_id) {
        this.productId = product_id;
    }

    public int getProductId() {
        return this.productId;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getCategory() {
        return this.category;
    }

    public void setProductName(String product_name) {
        this.productName = product_name;
    }
```

```
    public String getProductName() {
        return this.productName;
    }
}

interface Search {
    Product searchProduct(List<Product> products, int product_id);
}

class LinearSearch implements Search {
    public Product searchProduct(List<Product> products, int product_id) {
        for (Product p : products) {
            if (p.getProductId() == product_id) {
                return p;
            }
        }
        return null;
    }
}

class BinarySearch implements Search {
    public Product searchProduct(List<Product> products, int product_id) {
        int s = 0;
        int e = products.size() - 1;
        int mid = s + (e - s) / 2;

        while (e >= s) {
            if (product_id == products.get(mid).getProductId()) {
                return products.get(mid);
```

```
        } else if (product_id < products.get(mid).getProductId()) {
            e = mid - 1;
            mid = s + (e - s) / 2;
        } else {
            s = mid + 1;
            mid = s + (e - s) / 2;
        }
    }

    return null;
}

public String getName(List<Product> products, int product_id) {
    int s = 0;
    int e = products.size() - 1;
    int mid = s + (e - s) / 2;

    while (e >= s) {
        if (product_id == products.get(mid).getProductId()) {
            return products.get(mid).getProductName();
        } else if (product_id < products.get(mid).getProductId()) {
            e = mid - 1;
            mid = s + (e - s) / 2;
        } else {
            s = mid + 1;
            mid = s + (e - s) / 2;
        }
    }
```

```
        return "Product Not Found";
    }
}

class MergeSort {
    public static void sort(List<Product> products) {
        int s = 0;
        int e = products.size() - 1;
        divide(products, s, e);
    }

    private static void divide(List<Product> products, int s, int e) {
        if (e <= s) {
            return;
        }
        int mid = s + (e - s) / 2;
        divide(products, s, mid);
        divide(products, mid + 1, e);
        merge(products, s, mid, e);
    }

    private static void merge(List<Product> products, int s, int mid, int e) {
        int i = s;
        int j = mid + 1;

        List<Product> merged = new ArrayList<>();

        while (i <= mid && j <= e) {
            if (products.get(i).getProductId() <= products.get(j).getProductId()) {
```

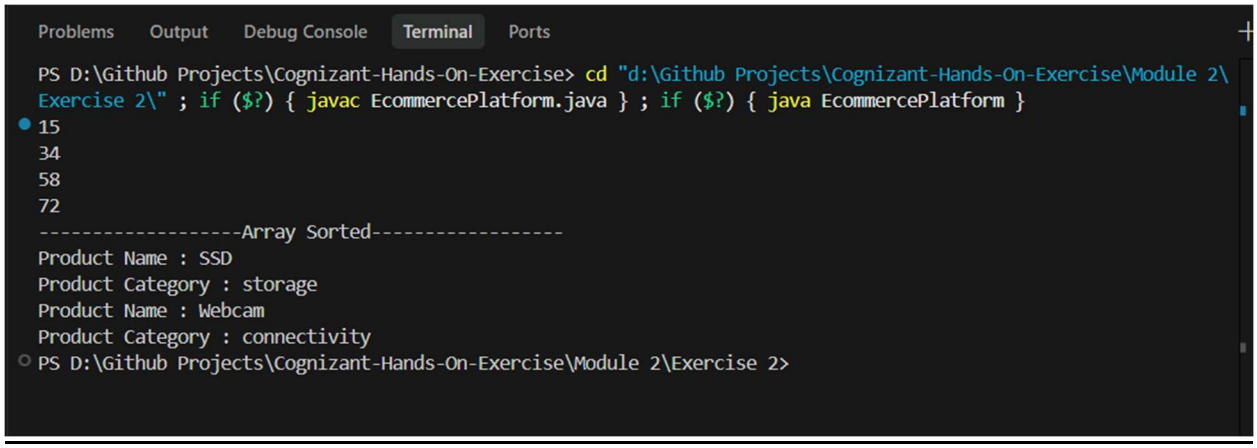
```
            merged.add(products.get(i++));
        } else {
            merged.add(products.get(j++));
        }
    }

    while (i <= mid) {
        merged.add(products.get(i++));
    }

    while (j <= e) {
        merged.add(products.get(j++));
    }

    int k = s;
    for (Product p : merged) {
        products.set(k++, p);
    }
  }
}
```

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
PS D:\Github Projects\Cognizant-Hands-On-Exercise> cd "d:\Github Projects\Cognizant-Hands-On-Exercise\Module 2\Exercise 2\" ; if ($?) { javac EcommercePlatform.java } ; if ($?) { java EcommercePlatform }
15
34
58
72
------------------Array Sorted-----------------
Product Name : SSD
Product Category : storage
Product Name : Webcam
Product Category : connectivity
PS D:\Github Projects\Cognizant-Hands-On-Exercise\Module 2\Exercise 2>